



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

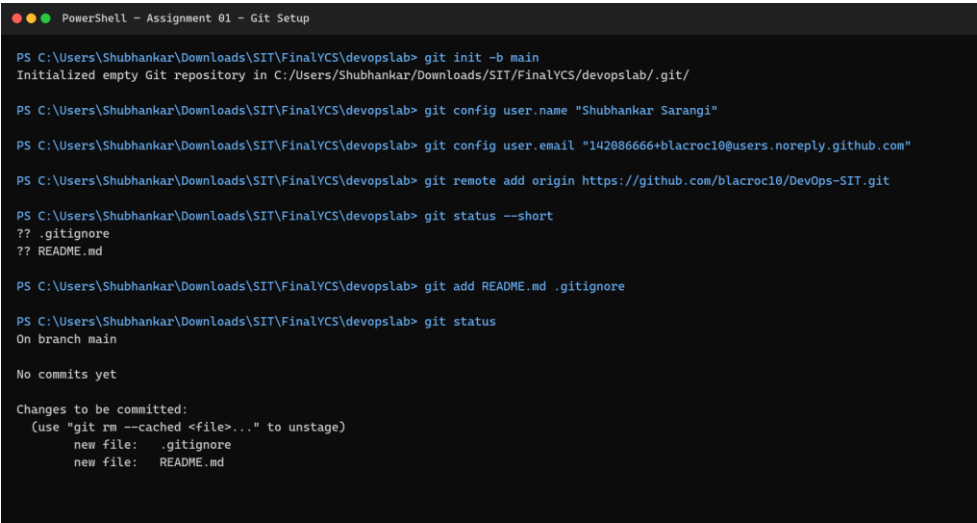
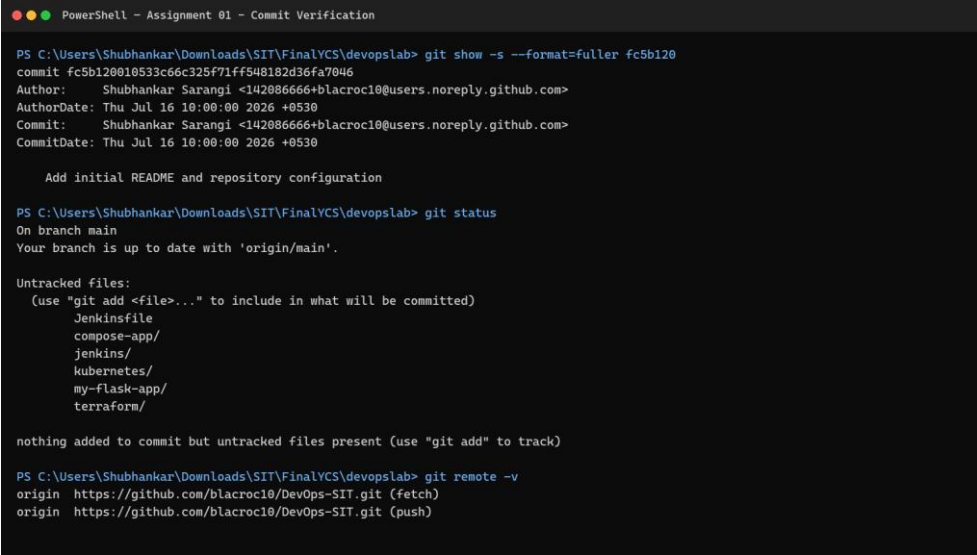
(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 01

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	16th July, 2026
Title of Assignment:	Git Setup & Basic Workflow
Theory:	Key concepts • Git is a distributed version control system that records source-code changes as commits and allows every clone to retain the complete project history. • The working tree contains edited files, the staging area selects the next snapshot, and the local repository stores committed snapshots. • GitHub hosts the remote repository so that committed work can be synchronized and reviewed from any authorized machine. • A clear commit message and a clean working tree make project history easier to audit and reproduce. Commands / configuration used • git init • git config user.name "Shubhankar Sarangi" • git add README.md .gitignore • git commit -m "Add initial README and repository configuration" • git remote add origin https://github.com/blacroc10/DevOps-SIT.git • git push -u origin main • git log --oneline Repository: https://github.com/blacroc10/DevOps-SIT
Implementation:	Workflow and evidence
Step 1	Initialize the repository and configure the remote The local repository, identity, initial files, and GitHub remote were configured.

	<pre>PowerShell - Assignment 01 - Git Setup PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git init -b main Initialized empty Git repository in C:/Users/Shubhankar/Downloads/SIT/FinalVCS/devopslab/.git/ PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git config user.name "Shubhankar Sarangi" PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git config user.email "142086666+blacroc10@users.noreply.github.com" PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git remote add origin https://github.com/blacroc10/DevOps-SIT.git PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git status --short ?? .gitignore ?? README.md PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git add README.md .gitignore PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git status On branch main No commits yet Changes to be committed: (use "git rm --cached <file>..." to unstage) new file: .gitignore new file: README.md</pre> Figure 1.1: Initialize the repository and configure the remote
Step 2	Commit and verify the basic Git workflow The files were staged and committed, and the clean working tree and commit history were verified. <pre>PowerShell - Assignment 01 - Commit Verification PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git show -s --format=fuller fc5b120 commit fc5b120810533c66c325f71ff548182d36fa7046 Author: Shubhankar Sarangi <142086666+blacroc10@users.noreply.github.com> AuthorDate: Thu Jul 16 10:00:00 2026 +0530 Commit: Shubhankar Sarangi <142086666+blacroc10@users.noreply.github.com> CommitDate: Thu Jul 16 10:00:00 2026 +0530 Add initial README and repository configuration PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git status On branch main Your branch is up to date with 'origin/main'. Untracked files: (use "git add <file>..." to include in what will be committed) Jenkinsfile compose-app/ jenkins/ kubernetes/ my-flask-app/ terraform/ nothing added to commit but untracked files present (use "git add" to track) PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git remote -v origin https://github.com/blacroc10/DevOps-SIT.git (fetch) origin https://github.com/blacroc10/DevOps-SIT.git (push)</pre> Figure 1.2: Commit and verify the basic Git workflow
Step 3	Verify commits on GitHub The GitHub commit history confirms that the repository was pushed successfully under the student's account.

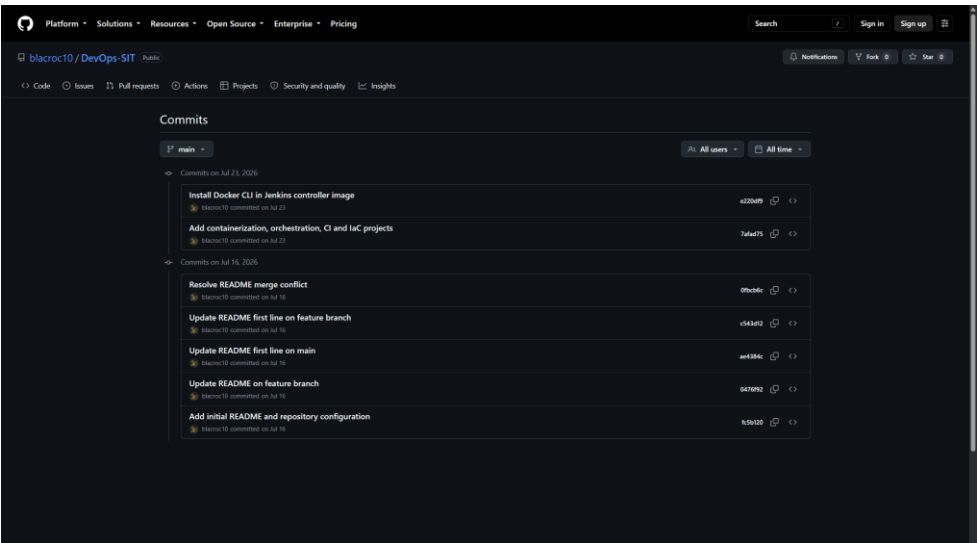


Figure 1.3: Verify commits on GitHub

Result:

The assignment workflow completed successfully, and the screenshots above were captured from the student's implementation.

Subject Faculty Name: Prof. Arijit Dutta



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 02

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	16th July, 2026
Title of Assignment:	Git Branching & Merging
Theory:	Key concepts • A Git branch is a movable pointer to a commit and provides an isolated line of development. • Feature branches allow changes to be developed and reviewed without destabilizing the main branch. • A merge combines the history and content of another branch into the currently checked-out branch. • Deleting a merged feature branch removes the temporary pointer without deleting its merged commits. Commands / configuration used • <code>git switch -c feature-update</code> • <code>git add README.md</code> • <code>git commit -m "Update README on feature branch"</code> • <code>git switch main</code> • <code>git merge feature-update</code> • <code>git branch -d feature-update</code> • <code>git log --oneline --graph --all</code>
Implementation:	Workflow and evidence
Step 1	Create and work on a feature branch A separate feature branch was created and used for the README update.

	PowerShell - Assignment 02 - Branching PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git status On branch main Your branch is up to date with 'origin/main'. nothing to commit, working tree clean PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git checkout -b feature-update PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git branch * feature-update main Figure 2.1: Create and work on a feature branch
Step 2	Commit the feature and merge it into main The feature commit was merged into main and the resulting branch history was verified. PowerShell - Assignment 02 - Merge PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git status --short M README.md PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git add README.md PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git commit -m "Update README on feature branch" [feature-update 0476f92] Update README on feature branch 1 file changed, 5 insertions(+) PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git checkout main Your branch is up to date with 'origin/main'. PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git merge feature-update Updating fc5b120..0476f92 Fast-forward README.md 5 +++++ 1 file changed, 5 insertions(+) PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git push origin main PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git log --oneline --graph --decorate -2 * 0476f92 (HEAD -> main, origin/main, feature-update) Update README on feature branch * fc5b120 Add initial README and repository configuration Figure 2.2: Commit the feature and merge it into main
Result:	The assignment workflow completed successfully, and the screenshots above were captured from the student's implementation.

Subject Faculty Name: Prof. Arijit Dutta



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

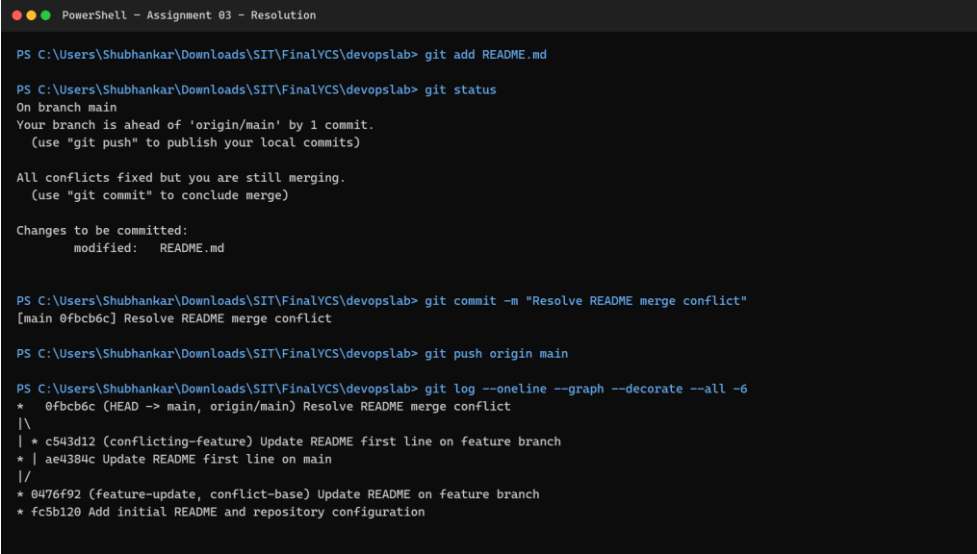
Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 03

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	16th July, 2026
Title of Assignment:	Simulating Git Collaboration & Conflict Resolution
Theory:	Key concepts • A merge conflict occurs when Git cannot safely combine competing edits to the same region of a file. • Conflict markers identify the current branch content, the separator, and the incoming branch content. • The developer must choose or combine the intended text, remove all conflict markers, stage the resolved file, and complete the merge commit. • Reviewing the final file and graph confirms that no edits were lost and both lines of development are represented. Commands / configuration used • git switch -c conflicting-feature • git switch main • git merge conflicting-feature • git status • git add README.md • git commit -m "Resolve README merge conflict" • git log --oneline --graph --all
Implementation:	Workflow and evidence
Step 1	Create a competing change on main The same README line was edited differently to simulate concurrent collaboration.

	PowerShell - Assignment 03 - Main Change <pre>PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git checkout main Your branch is up to date with 'origin/main'. PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git status On branch main Your branch is up to date with 'origin/main'. nothing to commit, working tree clean PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git branch conflict-base PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git diff -- README.md diff --git a/README.md b/README.md index e5596d3..3b2c799 100644 --- a/README.md +++ b/README.md @@ -1,4 +1,4 @@ -# DevOps Lab +# DevOps Lab (Main Branch Update) DevOps laboratory assignments completed by Shubhankar Sarangi (PRN: 23070122206). PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git add README.md PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git commit -m "Update README first line on main" [main ae4384c] Update README first line on main 1 file changed, 1 insertion(+), 1 deletion(-)</pre> Figure 3.1: Create a competing change on main
Step 2	PowerShell - Assignment 03 - Conflict (1/2) <pre>PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git checkout -b conflicting-feature conflict-base PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git branch --show-current conflicting-feature PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git diff -- README.md diff --git a/README.md b/README.md index e5596d3..946c531 100644 --- a/README.md +++ b/README.md @@ -1,4 +1,4 @@ -# DevOps Lab +# Awesome DevOps Lab (Feature Branch Update) DevOps laboratory assignments completed by Shubhankar Sarangi (PRN: 23070122206). PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git add README.md PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git commit -m "Update README first line on feature branch" [conflicting-feature c543d12] Update README first line on feature branch 1 file changed, 1 insertion(+), 1 deletion(-) PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git checkout main Your branch is ahead of 'origin/main' by 1 commit. (use "git push" to publish your local commits) PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git merge conflicting-feature Auto-merging README.md CONFLICT (content): Merge conflict in README.md Automatic merge failed; fix conflicts and then commit the result. PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git status --short UU README.md</pre> Figure 3.2: Attempt the merge and observe the conflict
Step 3	PowerShell - Assignment 03 - Conflict (2/2) <pre>PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> git diff -- README.md diff --cc README.md index 3b2c799,946c531..0000000 --- a/README.md +++ b/README.md @@@ -1,4 -1,4 +1,8 @@@ +<<<<<<< HEAD +# DevOps Lab (Main Branch Update) +<<<<<<<< + # Awesome DevOps Lab (Feature Branch Update) +>>>>>>>> conflicting-feature DevOps laboratory assignments completed by Shubhankar Sarangi (PRN: 23070122206).</pre> Figure 3.3: Inspect the conflict markers

Step 4	Resolve, commit, and verify the merge The final text was selected, staged, committed, and confirmed in the branch graph.  The screenshot shows a PowerShell terminal window titled "PowerShell - Assignment 03 - Resolution". The user is in the directory "C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab". They run "git add README.md", "git status", and "git push origin main". The output shows that the branch is ahead of 'origin/main' by 1 commit. They then run "git commit -m 'Resolve README merge conflict'", which shows the changes to be committed: "modified: README.md". Finally, they run "git log --oneline --graph --decorate --all -6", which shows the commit history with the merge conflict resolution. <i>Figure 3.4: Resolve, commit, and verify the merge</i>
Result:	The assignment workflow completed successfully, and the screenshots above were captured from the student's implementation.

Subject Faculty Name: Prof. Arijit Dutta



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 04

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	16th July, 2026
Title of Assignment:	Jira Project & Issue Tracking
Theory:	Key concepts • Jira is a work-management platform used to plan, track, prioritize, and report software tasks. • A Task represents a unit of work, a Story captures user-facing value, and a Bug records incorrect behavior. • A Kanban board visualizes work through workflow columns such as To Do, In Progress, and Done. • Moving work items between statuses provides a transparent history of progress and helps identify bottlenecks. Commands / configuration used • Create project: DevOps Lab Project (key: KAN) • Create Task: Set up database • Create Story: Develop login page • Create Bug: Submit button not working • Move each work item: To Do -> In Progress -> Done
Implementation:	Workflow and evidence
Step 1	Create Task, Story, and Bug work items The board shows all three requested work-item types in their initial workflow states.

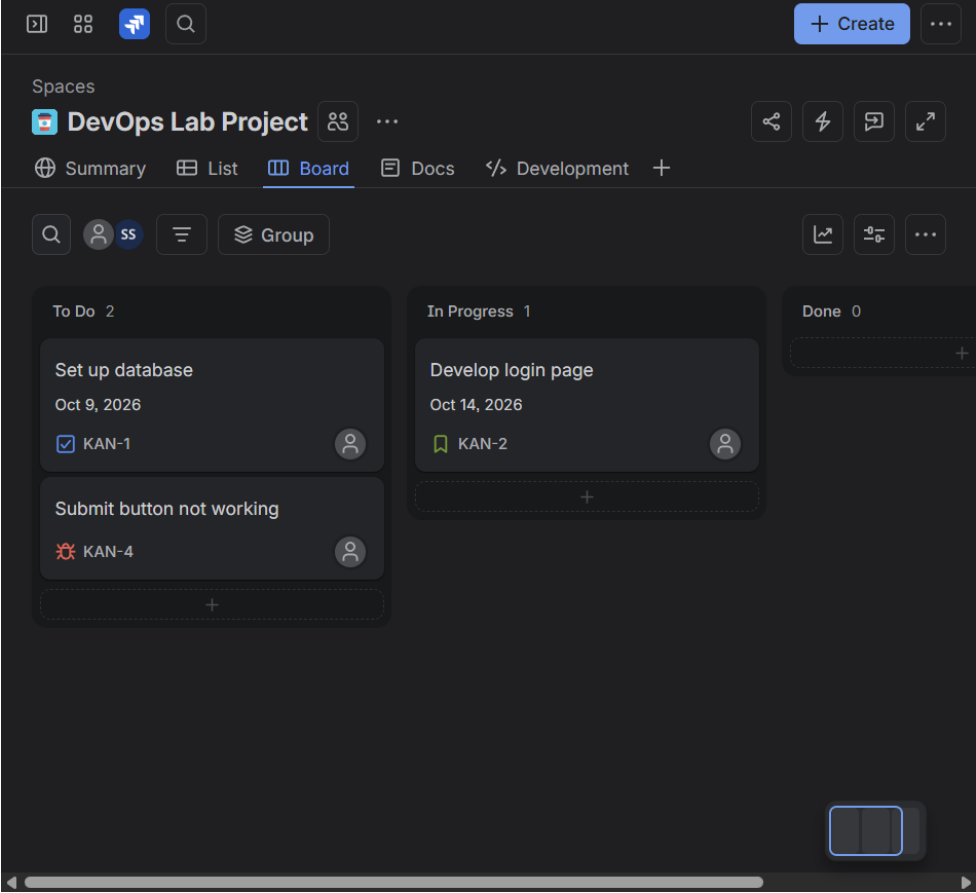


Figure 4.1: Create Task, Story, and Bug work items

Step 2

Move the work items to In Progress
All three work items were moved to the active work column.

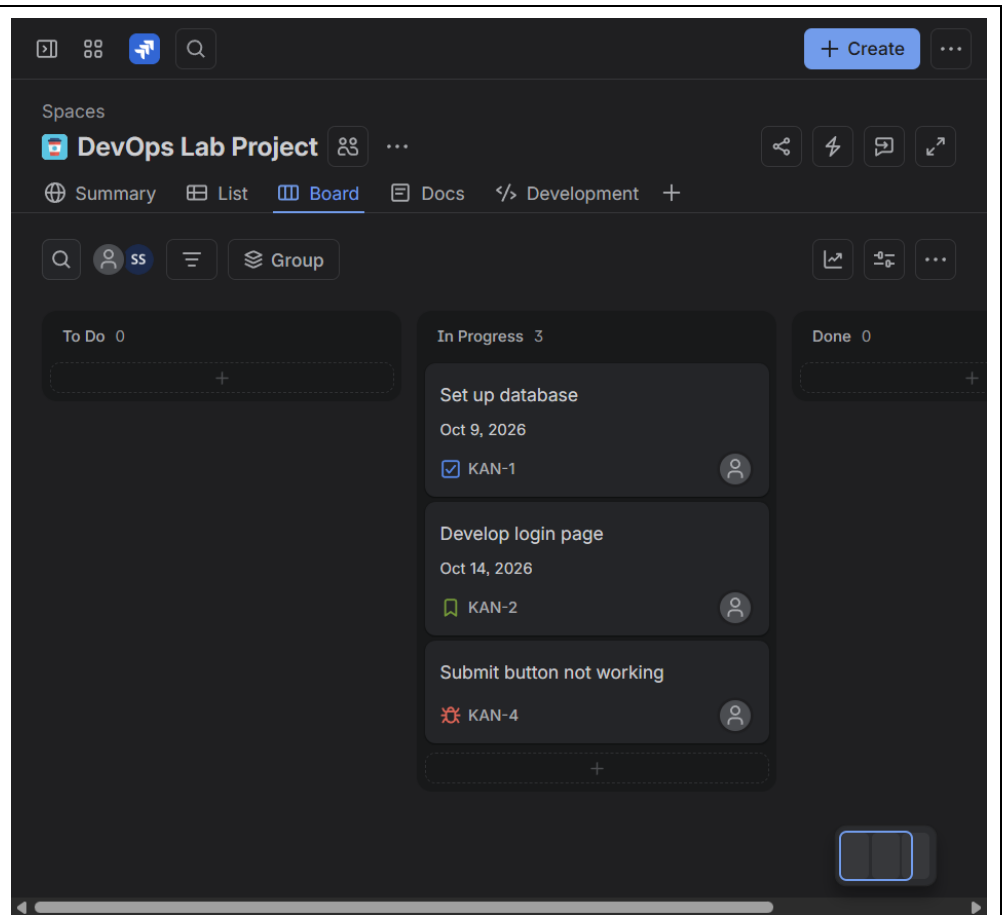


Figure 4.2: Move the work items to In Progress

Step 3

Complete the workflow

The Task, Story, and Bug were moved to Done, completing the Kanban workflow.

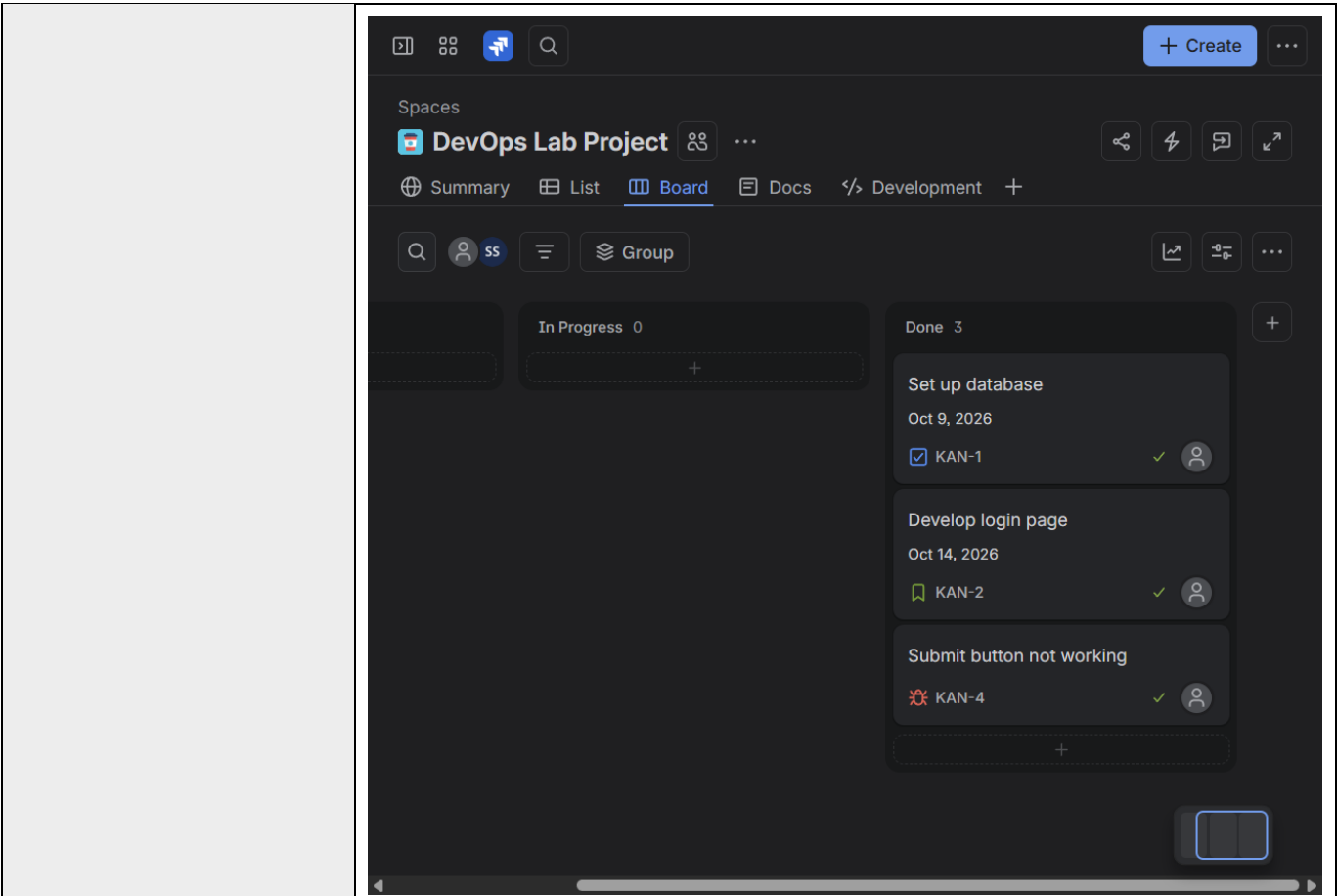


Figure 4.3: Complete the workflow

Result:

The assignment workflow completed successfully, and the screenshots above were captured from the student's implementation.

Subject Faculty Name: Prof. Arijit Dutta



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

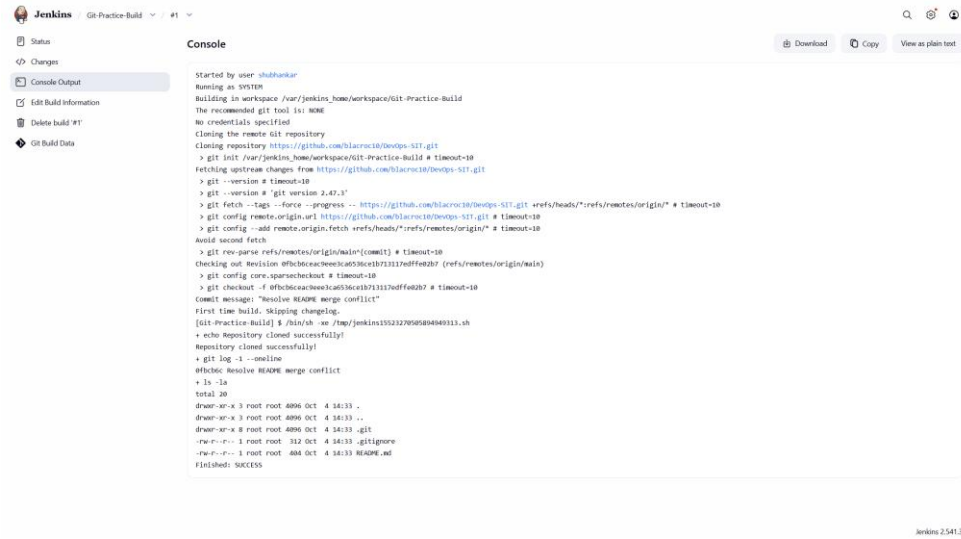
Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 05

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	16th July, 2026
Title of Assignment:	Continuous Integration with Jenkins
Theory:	Key concepts • Continuous Integration automatically builds and tests changes frequently so integration problems are discovered early. • Jenkins is an automation server that executes jobs on a controller or configured agents and records console output for each build. • A Freestyle project provides a graphical job configuration for source control, triggers, environment settings, and build steps. • A successful console log and green build status provide traceable evidence that the configured steps completed. Commands / configuration used • docker build -t devops-jenkins:lab jenkins • docker run -d --name devops-jenkins -p 8090:8080 devops-jenkins:lab • Create Freestyle project: devops-freestyle • Execute shell step: echo "Hello from Jenkins CI" • Build Now and inspect Console Output
Implementation:	Workflow and evidence
Step 1	Start the Jenkins controller container A dedicated Jenkins controller image and container were prepared for the lab.

	 PowerShell - Assignment 05 - Jenkins Setup <pre>PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> docker images devops-jenkins:lts-docker IMAGE ID DISK USAGE CONTENT SIZE EXTRA devops-jenkins:lts-docker 1be950035fac 1.24GB 418MB U PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> docker ps --filter name=devops-jenkins CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS aded899c803c devops-jenkins:lts-docker "/usr/bin/tini -- /uſſ*" 13 minutes ago Up 13 minutes 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp, 0.0.0.0:8090->8080/tcp, [::]:8090->8080/tcp devops-jenkins PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> docker exec devops-jenkins java -version PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> docker exec devops-jenkins git --version git version 2.47.3</pre> Figure 5.1: Start the Jenkins controller container
Step 2	 Getting Started Jenkins is ready! Your Jenkins setup is complete. Start using Jenkins Figure 5.2: Verify the Jenkins dashboard
Step 3	 PowerShell - Assignment 05 - Freestyle Build (1/2) <pre>PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> Jenkins API: create Git-Practice-Build Created Freestyle project with Git SCM: https://github.com/blacrocl0/DevOps-SIT.git Triggered build for Git-Practice-Build. devops-jenkins devops-jenkins:lts-docker Up 7 minutes 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp, 0.0.0.0:8090->8080/tcp, [::]:8090->8080/tcp Build #1: SUCCESS Started by user shubhankar Running as SYSTEM Building in workspace /var/jenkins_home/workspace/Git-Practice-Build The recommended git tool is: NONE No credentials specified Cloning the remote Git repository Cloning repository https://github.com/blacrocl0/DevOps-SIT.git > git init /var/jenkins_home/workspace/Git-Practice-Build # timeout=10 Fetching upstream changes from https://github.com/blacrocl0/DevOps-SIT.git > git --version # timeout=10 > git --version # 'git version 2.47.3' > git fetch --tags --force --progress -- https://github.com/blacrocl0/DevOps-SIT.git +refs/heads/*:refs/remotes/origin/* # timeout=10 > git config remote.origin.url https://github.com/blacrocl0/DevOps-SIT.git # timeout=10 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10 Avoid second fetch > git rev-parse refs/remotes/origin/main^{commit} # timeout=10 Checking out Revision 0fbcb6ceac9eee3ca6536ce1b713117edffe02b7 (refs/remotes/origin/main) > git config core.sparsecheckout # timeout=10 > git checkout -f 0fbcb6ceac9eee3ca6536ce1b713117edffe02b7 # timeout=10 Commit message: "Resolve README merge conflict" First time build. Skipping changelog. [Git-Practice-Build] \$ /bin/sh -xe /tmp/jenkins15523270505894949313.sh + echo Repository cloned successfully! Repository cloned successfully! + git log -1 --oneline 0fbcb6c Resolve README merge conflict + ls -la</pre> Figure 5.3: Run the Freestyle job
Step 4	 Inspect the completed console log

	The build log shows the command output and successful completion. <pre>total 20 drwxr-xr-x 3 root root 4096 Oct 4 14:33 . drwxr-xr-x 3 root root 4096 Oct 4 14:33 .. drwxr-xr-x 8 root root 4096 Oct 4 14:33 .git -rw-r--r-- 1 root root 312 Oct 4 14:33 .gitignore -rw-r--r-- 1 root root 404 Oct 4 14:33 README.md Finished: SUCCESS</pre> Figure 5.4: Inspect the completed console log
Step 5	Confirm the successful build The job dashboard records the build as successful.  Figure 5.5: Confirm the successful build
Result:	The assignment workflow completed successfully, and the screenshots above were captured from the student's implementation.

Subject Faculty Name: Prof. Arijit Dutta



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

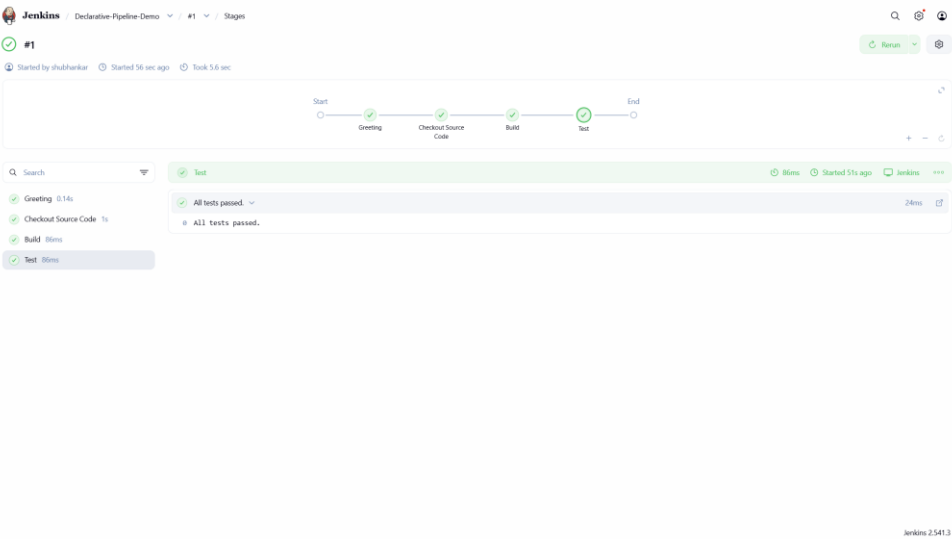
Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 06

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	16th July, 2026
Title of Assignment:	Basic Declarative Pipeline
Theory:	Key concepts • Pipeline as Code stores the CI process in version control so it can be reviewed and reproduced. • A declarative Jenkins pipeline uses a pipeline block containing an agent, stages, and individual steps. • Stages divide the workflow into understandable phases such as Build, Test, and Deploy. • Jenkins records each stage independently, making failures and execution duration easy to identify. Commands / configuration used • <code>pipeline { agent any; stages { ... } }</code> • <code>stage('Build') { steps { echo 'Building...' } }</code> • <code>stage('Test') { steps { echo 'Testing...' } }</code> • <code>stage('Deploy') { steps { echo 'Deploying...' } }</code> • Run the pipeline and inspect Stage View
Implementation:	Workflow and evidence
Step 1	Execute the declarative pipeline Jenkins parsed the pipeline and began executing the declared stages.

	PowerShell - Assignment 06 - Declarative Pipeline (1/2) <pre>PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> Jenkins API: create Declarative-Pipeline-Demo Created pipeline stages: Greeting, Checkout Source Code, Build, Test. Triggered Declarative-Pipeline-Demo build. Build #1: SUCCESS Started by user shubhankar [Pipeline] Start of Pipeline [Pipeline] node Running on Jenkins in /var/jenkins_home/workspace/Declarative-Pipeline-Demo [Pipeline] { [Pipeline] stage [Pipeline] { (Greeting) [Pipeline] echo Hello from the DevOps Lab pipeline! [Pipeline] } [Pipeline] // stage [Pipeline] stage [Pipeline] { (Checkout Source Code) [Pipeline] git The recommended git tool is: NONE No credentials specified Cloning the remote Git repository Cloning repository https://github.com/blacroc10/DevOps-SIT.git > git init /var/jenkins_home/workspace/Declarative-Pipeline-Demo # timeout=10 Fetching upstream changes from https://github.com/blacroc10/DevOps-SIT.git > git --version # timeout=10 > git --version 'git version 2.47.3' > git fetch --tags --force --progress -- https://github.com/blacroc10/DevOps-SIT.git +refs/heads/*:refs/remotes/origin/* # timeout=10 > git config remote.origin.url https://github.com/blacroc10/DevOps-SIT.git # timeout=10 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10 Avoid second fetch > git rev-parse refs/remotes/origin/main^{commit} # timeout=10 Checking out Revision 0fbcb6ceac9eee3ca6536ce1b713117edffe02b7 (refs/remotes/origin/main) > git config core.sparsecheckout # timeout=10</pre> Figure 6.1: Execute the declarative pipeline
Step 2	Verify stage output in the console The build, test, and deployment messages completed without errors. PowerShell - Assignment 06 - Declarative Pipeline (2/2) <pre>> git checkout -f 0fbcb6ceac9eee3ca6536ce1b713117edffe02b7 # timeout=10 > git branch -a -v --no-abbrev # timeout=10 > git checkout -b main 0fbcb6ceac9eee3ca6536ce1b713117edffe02b7 # timeout=10 Commit message: "Resolve README merge conflict" First time build. Skipping changelog. [Pipeline] } [Pipeline] // stage [Pipeline] stage [Pipeline] { (Build) [Pipeline] echo Building the application... [Pipeline] } [Pipeline] // stage [Pipeline] stage [Pipeline] { (Test) [Pipeline] echo All tests passed. [Pipeline] } [Pipeline] // stage [Pipeline] } [Pipeline] // node [Pipeline] End of Pipeline Finished: SUCCESS</pre> Figure 6.2: Verify stage output in the console
Step 3	Review the pipeline stage view The visual stage view confirms that every stage completed successfully.

	The screenshot shows the Jenkins web interface for a pipeline named 'Declarative Pipeline-Demo'. The pipeline is in a 'Stages' view, showing a sequence of four stages: 'Greeting', 'Checkout Source Code', 'Build', and 'Test'. All stages are marked with green checkmarks, indicating they completed successfully. The 'Test' stage is currently selected, showing a summary of 'All tests passed.' with a duration of 24ms. On the left, a search bar and a list of stages with their durations are visible: 'Greeting' (0.14s), 'Checkout Source Code' (1s), 'Build' (80ms), and 'Test' (86ms). The top right corner includes a 'Run' button and a user profile icon. The bottom right corner of the interface displays the version 'Jenkins 2.541.3'. <i>Figure 6.3: Review the pipeline stage view</i>
--	---

Subject Faculty Name: Prof. Arijit Dutta



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 07

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	Pipeline with Parameters
Theory:	Key concepts • Pipeline parameters allow a single Jenkins job to accept user input at build time. • String, choice, Boolean, and password parameters can control environments, versions, or optional behavior. • Declarative pipelines define parameters in a parameters block and access values through the params object. • Parameterized builds improve reuse because the pipeline logic does not need to be edited for every execution. Commands / configuration used • <code>parameters { string(name: 'GREETING_NAME', defaultValue: 'Shubhankar') }</code> • <code>echo "Hello \${params.GREETING_NAME}"</code> • Select Build with Parameters • Enter a parameter value and start the build
Implementation:	Workflow and evidence
Step 1	Configure and start a parameterized build The build was started with an explicit parameter value.

	 PowerShell - Assignment 07 - Parameters (1/2) <pre>PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> Jenkins API: add GREETING_NAME parameter Configured string parameter GREETING_NAME with default value World. Triggered parameterized build with GREETING_NAME=Shubhankar. Ran the updated pipeline once to register its parameter definition. Triggered parameterized build with GREETING_NAME=Shubhankar. Build #3: SUCCESS Started by user shubhankar [Pipeline] Start of Pipeline [Pipeline] node Running on Jenkins in /var/jenkins_home/workspace/Declarative-Pipeline-Demo [Pipeline] { [Pipeline] stage [Pipeline] { (Greeting) [Pipeline] echo Hello Shubhankar! [Pipeline] } [Pipeline] // stage [Pipeline] stage [Pipeline] { (Checkout Source Code) [Pipeline] git The recommended git tool is: NONE No credentials specified > git rev-parse --resolve-git-dir /var/jenkins_home/workspace/Declarative-Pipeline-Demo/.git # timeout=10 Fetching changes from the remote Git repository > git config remote.origin.url https://github.com/blacroc10/DevOps-SIT.git # timeout=10 Fetching upstream changes from https://github.com/blacroc10/DevOps-SIT.git > git --version # timeout=10 > git --version # 'git version 2.47.3' > git fetch --tags --force --progress -- https://github.com/blacroc10/DevOps-SIT.git +refs/heads/*:refs/remotes/origin/* # timeout=10 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10 Checking out Revision 0fbcb6ceac9eee3ca6536ce1b713117edffe02b7 (refs/remotes/origin/main) > git config core.sparsecheckout # timeout=10 > git checkout -f 0fbcb6ceac9eee3ca6536ce1b713117edffe02b7 # timeout=10</pre>
Step 2	 PowerShell - Assignment 07 - Parameters (2/2) <pre>> git branch -a -v --no-abbrev # timeout=10 > git branch -D main # timeout=10 > git checkout -b main 0fbcb6ceac9eee3ca6536ce1b713117edffe02b7 # timeout=10 Commit message: "Resolve README merge conflict" > git rev-list --no-walk 0fbcb6ceac9eee3ca6536ce1b713117edffe02b7 # timeout=10 [Pipeline] } [Pipeline] // stage [Pipeline] stage [Pipeline] { (Build) [Pipeline] echo Building the application... [Pipeline] } [Pipeline] // stage [Pipeline] stage [Pipeline] { (Test) [Pipeline] echo All tests passed. [Pipeline] } [Pipeline] // stage [Pipeline] } [Pipeline] // node [Pipeline] End of Pipeline Finished: SUCCESS</pre>

Figure 7.2: Execute the parameter-aware pipeline

	Jenkins Declarative Pipeline Demo #3 Status Changes Console Output Edit Build Information Delete build #3 Parameters Git Build Data Pipeline Overview Restart from Stage Replay Pipeline Steps Workspaces Previous Build Console Download Copy View as plain text Started by user shubhankar [Pipeline] start of Pipeline [Pipeline] node Running on Jenkins in /var/jenkins_home/workspaces/Declarative-Pipeline-Demo [Pipeline] { [Pipeline] stage [Pipeline] { (greeting) [Pipeline] echo hello shubhankar! [Pipeline] } [Pipeline] // stage [Pipeline] stage [Pipeline] { (checkout Source code) [Pipeline] git The recommended git tool is: NONE No credentials specified > git rev-parse --resolve-git-dir /var/jenkins_home/workspaces/Declarative-Pipeline-Demo/.git # timeout=10 Fetching changes from the remote git repository > git config remote.origin.url https://github.com/blacrc18/DevOps-SIT.git # timeout=10 Fetching upstream changes from https://github.com/blacrc18/DevOps-SIT.git > git --version # timeout=10 > git --version # 'git version 2.47.3' > git fetch --tags --force --progress -- https://github.com/blacrc18/DevOps-SIT.git +refs/heads/*:refs/remotes/origin/* # timeout=10 > git rev-parse refs/remotes/origin/main: (commit) # timeout=10 Checking out Revision 0fbcbceacbee3ca536ce3b733137d0ff6e2b7 (refs/remotes/origin/main) > git config core.sparsecheckout # timeout=10 > git checkout -f 0fbcbceacbee3ca536ce3b733137d0ff6e2b7 # timeout=10 > git branch -a -v --no-abbrev # timeout=10 > git branch -D main # timeout=10 > git checkout -b main 0fbcbceacbee3ca536ce3b733137d0ff6e2b7 # timeout=10 Commit message: "resolve ALAPM merge conflict" > git rev-list --no-walk 0fbcbceacbee3ca536ce3b733137d0ff6e2b7 # timeout=10 [Pipeline] } [Pipeline] // stage [Pipeline] stage [Pipeline] { (build) [Pipeline] echo Building the application... [Pipeline] } [Pipeline] // stage
Result:	The assignment workflow completed successfully, and the screenshots above were captured from the student's implementation.

Subject Faculty Name: Prof. Arijit Dutta



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 08

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	Dockerfile & Image Build
Theory:	Key concepts • A Dockerfile is a version-controlled recipe for building an immutable container image. • Instructions such as FROM, WORKDIR, COPY, RUN, EXPOSE, and CMD create ordered image layers. • An image packages the application and dependencies, while a container is a running instance of that image. • Port publishing maps a host port to the application port inside the container. Commands / configuration used • <code>docker build -t devops-sit-flask:1.0 my-flask-app</code> • <code>docker image ls devops-sit-flask</code> • <code>docker run -d --name flask-lab -p 8081:5000 devops-sit-flask:1.0</code> • <code>docker ps</code> • Open <code>http://localhost:8081</code>
Implementation:	Workflow and evidence
Step 1	Build and run the Flask image Docker built the application image and started a container from it.

	PowerShell - Assignment 08 - Docker Image Build <pre>PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> docker build -t devops-sit-flask:1.0 my-flask-app PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> docker images devops-sit-flask:1.0 IMAGE ID DISK USAGE CONTENT SIZE EXTRA devops-sit-flask:1.0 699705c1e1b0 209MB 51.2MB PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> docker run -d --name flask-build-demo -p 8083:5000 devops-sit-flask:1.0 e25c2edbfc2bda724afd2934f09680947ea9c9c6e91961b1cbb67f6b689c92 PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> docker ps --filter name=flask-build-demo CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS e25c2edbfc2bda724afd2934f09680947ea9c9c6e91961b1cbb67f6b689c92 devops-sit-flask:1.0 "python app.py" Less than a second ago Up Less than a second 0.0.0.0:8083->5000/tcp, [::]:8083->5000/tcp flask-build-demo PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> curl.exe -s http://localhost:8083/ flask-build-demo PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> docker rm -f flask-build-demo PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> docker build -t devops-sit-flask:1.0 my-flask-app PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> docker run -d --name flask-build-demo -p 8083:5000 devops-sit-flask:1.0 c4b19d5b9cb7281052be621fc82512c80b7a466f9f8a02a60e18e1fa39eb1d55</pre> Figure 8.1: Build and run the Flask image
Step 2	Verify the containerized application The Flask response was reached through the published host port. Hello, Docker! DevOps Lab application by Shubhankar Sarangi. Persistent visit count: 1 Figure 8.2: Verify the containerized application
Result:	The assignment workflow completed successfully, and the screenshots above were captured from the student's implementation.

Subject Faculty Name: Prof. Arijit Dutta



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

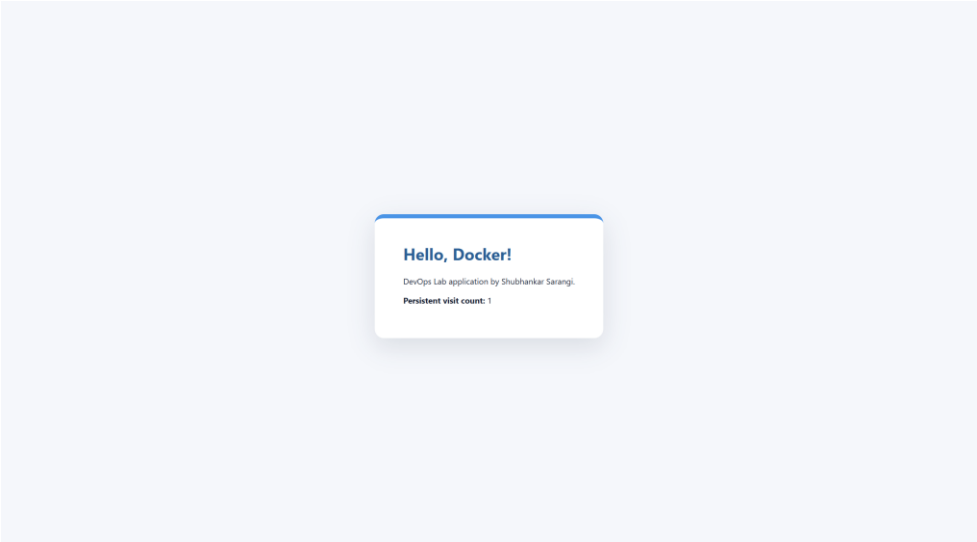
(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 09

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	Container Management & Networking
Theory:	Key concepts • Docker manages containers through a lifecycle of create, start, inspect, stop, and remove operations. • A user-defined bridge network provides automatic DNS resolution between containers by container name. • Network isolation limits communication to attached containers while published ports expose selected services to the host. • Inspection and connectivity tests verify addresses, attached containers, and application reachability. Commands / configuration used • <code>docker network create devops-network</code> • <code>docker run -d --name flask-networked --network devops-network -p 8083:5000 devops-sit-flask:1.0</code> • <code>docker network inspect devops-network</code> • <code>docker exec flask-networked hostname</code> • <code>docker ps</code>
Implementation:	Workflow and evidence
Step 1	Create and inspect the Docker network The application container was attached to a dedicated user-defined bridge network.

	PowerShell - Assignment 09 - Container Networking <pre>PS C:\Users\Shubhankar\Downloads\SIT\FinalYCS\devopslab> docker rm -f flask-build-demo flask-build-demo PS C:\Users\Shubhankar\Downloads\SIT\FinalYCS\devopslab> docker run -d --name flask-network-demo -p 8083:5000 devops-sit-flask:1.0 89ff222f43b8367573fabf91615962be979b70e02c4ed82883237d5ff5740bd2 PS C:\Users\Shubhankar\Downloads\SIT\FinalYCS\devopslab> docker ps --filter name=flask-network-demo CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES 89ff222f43b8 devops-sit-flask:1.0 "python app.py" 1 second ago Up Less than a second 0.0.0.0:8083->5000/tcp, [::]:8083->5000/tcp flask-network-demo PS C:\Users\Shubhankar\Downloads\SIT\FinalYCS\devopslab> docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}} Ports={{json .NetworkSettings.Ports}}' flask-network-demo IP=172.17.0.3 Ports={"5000/tcp":[{"HostIp":"0.0.0.0","HostPort":"8083"}, {"HostIp":":::", "HostPort":"8083"}]}</pre> <i>Figure 9.1: Create and inspect the Docker network</i>
Step 2	Verify the networked container The published service remained reachable from the host browser.  <i>Figure 9.2: Verify the networked container</i>
Result:	The assignment workflow completed successfully, and the screenshots above were captured from the student's implementation.

Subject Faculty Name: Prof. Arijit Dutta



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

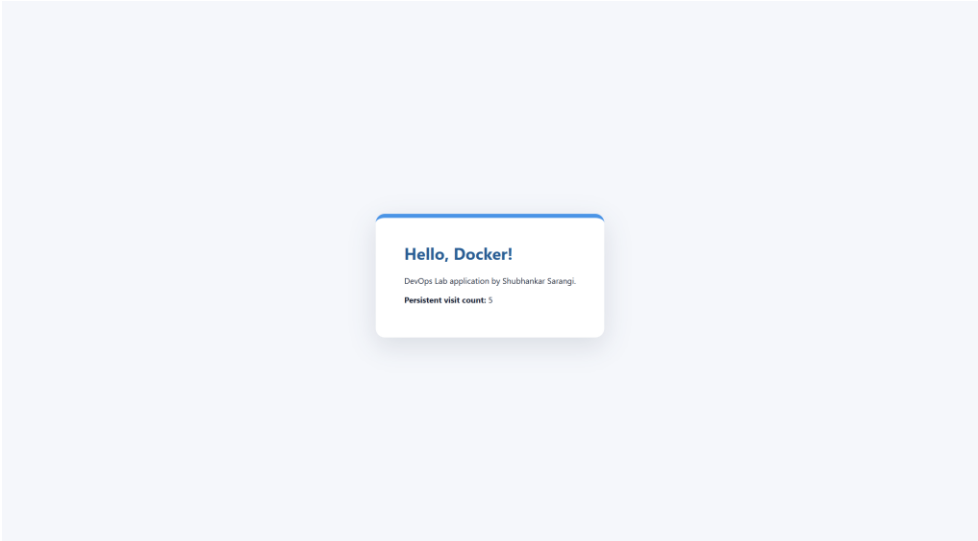
(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 10

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	Docker Volumes & Data Persistence
Theory:	Key concepts • Container writable layers are ephemeral and are removed with the container. • A Docker volume stores data independently of a container and can be mounted by replacement containers. • Named volumes are managed by Docker and are suitable for application state such as databases and uploaded files. • Persistence is verified by writing data, removing the original container, and reading the same data from a new container. Commands / configuration used • <code>docker volume create devops-data</code> • <code>docker run --rm -v devops-data:/data alpine sh -c "echo persistent-data > /data/message.txt"</code> • <code>docker run --rm -v devops-data:/data alpine cat /data/message.txt</code> • <code>docker volume inspect devops-data</code>
Implementation:	Workflow and evidence
Step 1	Create and test a named volume Data was written to the volume and recovered from a separate container.

	 PowerShell - Assignment 10 - Volumes <pre>PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> docker ps --filter name=flask-volume-demo-2 CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS 8eed36856671 devops-sit-flask:1.0 "python app.py" 28 seconds ago Up 28 seconds 0.0.0.0:8084->5000/tcp, [::]:8084->5000/tcp flask-volume-demo-2 PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> docker inspect -f "{{range .Mounts}}{{.Name}} -> {{.Destination}}{{end}}" flask-volume-demo-2 devops_flask_data -> /app/data PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> Invoke-WebRequest http://localhost:8084/ Persistent visit count after container recreation: 4 PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> docker exec flask-volume-demo-2 tail -n 4 /app/data/visits.log Visited at 2026-10-04T14:44:05.795758+00:00 Visited at 2026-10-04T14:44:05.847470+00:00 Visited at 2026-10-04T14:44:05.887188+00:00 Visited at 2026-10-04T14:44:37.985153+00:00</pre>
Step 2	 Verify the persistent application result The application workflow demonstrated that state remained available independently of the container.  Figure 10.2: Verify the persistent application result
Result:	 The assignment workflow completed successfully, and the screenshots above were captured from the student's implementation.

Subject Faculty Name: Prof. Arijit Dutta



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

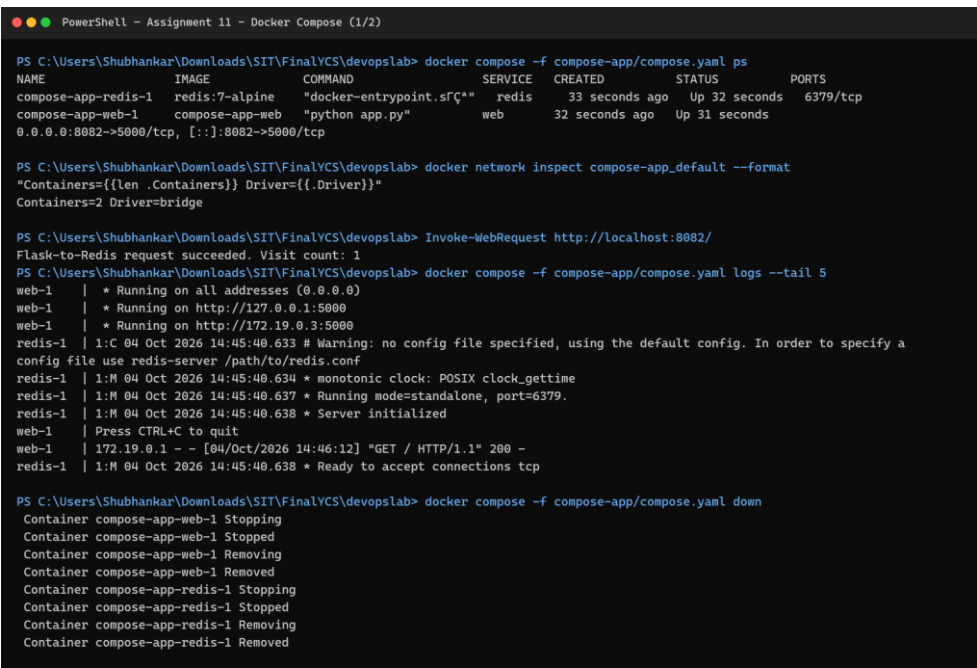
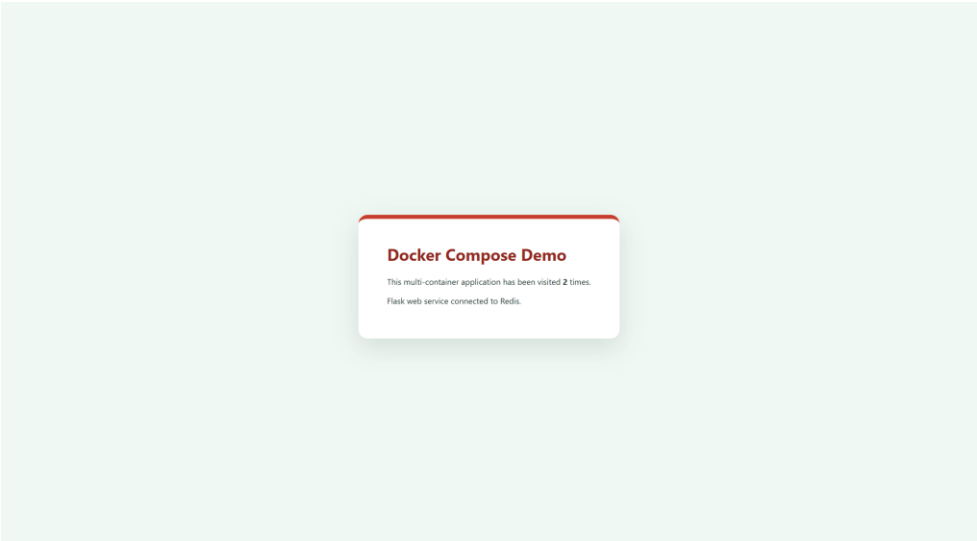
(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 11

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	Docker Compose for Multi-Container App
Theory:	Key concepts • Docker Compose defines a multi-container application in one YAML file. • Services describe images, builds, ports, environment variables, dependencies, networks, and volumes. • Compose creates a shared default network so the Flask service can resolve the Redis service by name. • A named Redis volume preserves data while docker compose up and down control the complete application stack. Commands / configuration used • <code>docker compose -f compose-app/compose.yaml up -d --build</code> • <code>docker compose -f compose-app/compose.yaml ps</code> • <code>docker compose -f compose-app/compose.yaml logs</code> • Open <code>http://localhost:8082</code> • <code>docker compose -f compose-app/compose.yaml down</code>
Implementation:	Workflow and evidence
Step 1	Build the Compose services Compose built the Flask service and prepared the Redis dependency.

	 Figure 11.1: Build the Compose services
Step 2	Start and inspect the service stack Both services reached the running state on the shared Compose network.  Figure 11.2: Start and inspect the service stack
Step 3	Verify Flask and Redis integration The browser response confirms successful communication between the web and Redis containers.  Figure 11.3: Verify Flask and Redis integration
Result:	The assignment workflow completed successfully, and the screenshots above were captured from the student's implementation.

Subject Faculty Name: Prof. Arijit Dutta



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 12

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	Kubernetes Pod & Deployment
Theory:	Key concepts • A Pod is Kubernetes' smallest deployable unit and contains one or more tightly coupled containers. • A Deployment manages a desired number of identical Pod replicas through a ReplicaSet. • Labels and selectors associate the Deployment with its Pods, while readiness probes control traffic eligibility. • Declarative manifests make the desired cluster state reviewable, repeatable, and self-healing. Commands / configuration used • <code>kind create cluster --name devops-lab</code> • <code>kind load docker-image devops-sit-flask:1.0 --name devops-lab</code> • <code>kubectl apply -f kubernetes/deployment.yaml</code> • <code>kubectl get deployments,pods -o wide</code> • <code>kubectl describe deployment flask-app-deployment</code>
Implementation:	Workflow and evidence
Step 1	Create the cluster and apply the Deployment The application image was loaded into Kind and the Deployment manifest was applied.

	PowerShell - Assignment 12 - Kubernetes Deployment (1/2) <pre>PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> & 'C:\Users\Shubhankar\AppData\Local\DevOpsLab\bin\kind.exe' create cluster --name devops-lab PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> & 'C:\Users\Shubhankar\AppData\Local\DevOpsLab\bin\kind.exe' load docker-image devops-sit-flask:1.0 --name devops-lab PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> kubectl config use-context kind-devops-lab Switched to context "kind-devops-lab". PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> kubectl apply -f kubernetes/deployment.yaml deployment.apps/flask-app-deployment created PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> kubectl rollout status deployment/flask-app-deployment --timeout=120s Waiting for deployment "flask-app-deployment" rollout to finish: 0 of 2 updated replicas are available... Waiting for deployment "flask-app-deployment" rollout to finish: 1 of 2 updated replicas are available... deployment "flask-app-deployment" successfully rolled out PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> kubectl get deployments,pods -o wide NAME READY UP-TO-DATE AVAILABLE AGE CONTAINERS IMAGES SELECTOR deployment.apps/flask-app-deployment 2/2 2 2 9s flask-app devops-sit-flask:1.0 app=flask-app NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES pod/flask-app-deployment-6f7c98968-dfckp 1/1 Running 0 9s 10.244.0.6 devops-lab-control-plane <none> pod/flask-app-deployment-6f7c98968-h5gs7 1/1 Running 0 9s 10.244.0.4 devops-lab-control-plane <none> PS C:\Users\Shubhankar\Downloads\SIT\FinalVCS\devopslab> kubectl logs deployment/flask-app-deployment --tail=8 * Running on http://10.244.0.6:5000 D[33mPress CTRL+C to quitD[0m 10.244.0.1 - - [04/Oct/2026 14:50:37] "GET /health HTTP/1.1" 200 -</pre>
Step 2	Verify Deployment replicas and Pods Kubernetes reported the desired replicas as available and ready. PowerShell - Assignment 12 - Kubernetes Deployment (2/2) <pre>10.244.0.1 - - [04/Oct/2026 14:50:42] "GET /health HTTP/1.1" 200 - 10.244.0.1 - - [04/Oct/2026 14:50:47] "GET /health HTTP/1.1" 200 - 10.244.0.1 - - [04/Oct/2026 14:50:52] "GET /health HTTP/1.1" 200 - 10.244.0.1 - - [04/Oct/2026 14:50:57] "GET /health HTTP/1.1" 200 - 10.244.0.1 - - [04/Oct/2026 14:51:02] "GET /health HTTP/1.1" 200 -</pre>
Result:	The assignment workflow completed successfully, and the screenshots above were captured from the student's implementation.

Subject Faculty Name: Prof. Arijit Dutta



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

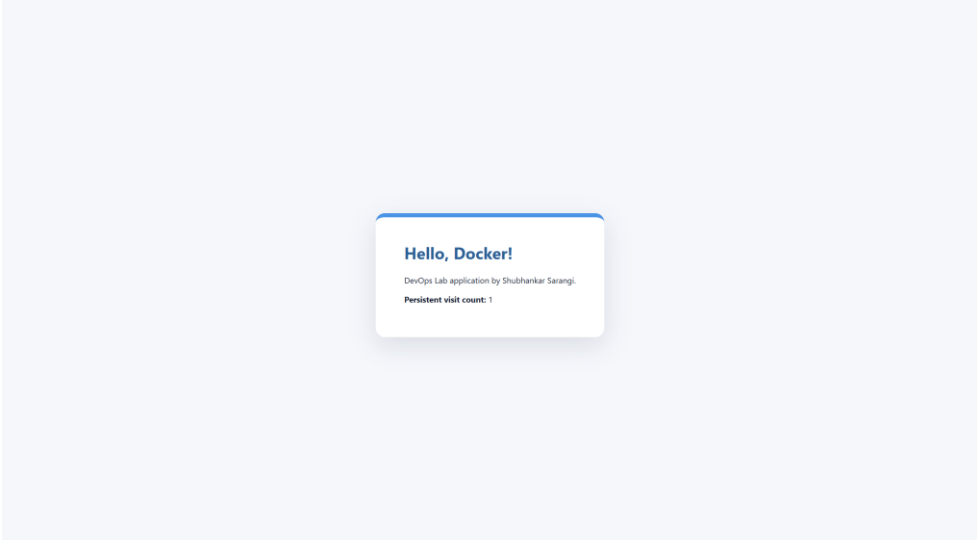
(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 13

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	Kubernetes Service & Networking
Theory:	Key concepts • Pods are replaceable, so a Kubernetes Service provides a stable virtual IP and DNS name. • A Service selector discovers matching Pods and load-balances traffic across their endpoints. • port is the Service-facing port, targetPort is the container port, and NodePort exposes a fixed port on each node. • Endpoints and an HTTP request verify that the Service routes traffic to ready application Pods. Commands / configuration used • <code>kubectl apply -f kubernetes/service.yaml</code> • <code>kubectl get service flask-app-service</code> • <code>kubectl get endpoints flask-app-service</code> • <code>kubectl port-forward service/flask-app-service 8084:80</code> • Open <code>http://localhost:8084</code>
Implementation:	Workflow and evidence
Step 1	Create and inspect the Kubernetes Service The NodePort Service selected the Flask Pods and exposed port 80 to the cluster.

	PowerShell - Assignment 13 - Kubernetes Service <pre>PS C:\Users\Shubhankar\Downloads\SIT\FinalYCS\devopslab> kubectl apply -f kubernetes/service.yaml service/flask-app-service created PS C:\Users\Shubhankar\Downloads\SIT\FinalYCS\devopslab> kubectl get service flask-app-service -o wide NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE SELECTOR flask-app-service NodePort 10.96.169.227 <none> 80:30080/TCP 0s app=flask-app PS C:\Users\Shubhankar\Downloads\SIT\FinalYCS\devopslab> kubectl get endpoints flask-app-service NAME ENDPOINTS AGE flask-app-service 10.244.0.4:5000,10.244.0.6:5000 0s</pre> <i>Figure 13.1: Create and inspect the Kubernetes Service</i>
Step 2	Verify Service routing A browser request routed through the Service and reached the deployed Flask application.  <i>Figure 13.2: Verify Service routing</i>
Result:	The assignment workflow completed successfully, and the screenshots above were captured from the student's implementation.

Subject Faculty Name: Prof. Arijit Dutta



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 14

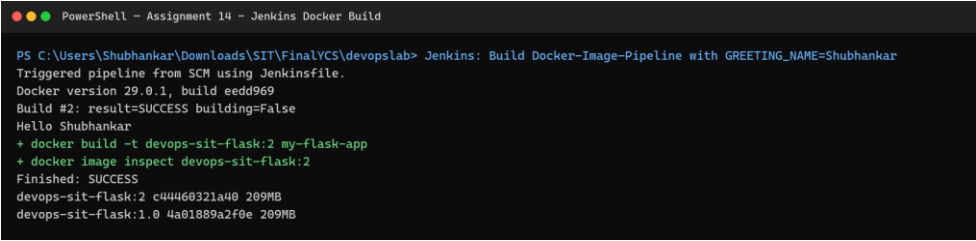
Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	CI/CD: Jenkins Pipeline Building a Docker Image
Theory:	Key concepts • CI/CD combines automated integration checks with repeatable delivery steps. • The Jenkins controller image includes the Docker CLI and uses the host Docker socket to invoke the Docker engine. • The version-controlled Jenkinsfile checks out the repository, builds a uniquely tagged image, and validates it. • A successful pipeline links a source revision, Jenkins build number, and produced Docker image for traceability. Commands / configuration used • checkout scm • docker build -t devops-sit-flask:\${BUILD_NUMBER} my-flask-app • docker image inspect devops-sit-flask:\${BUILD_NUMBER} • Run the Jenkins pipeline from the repository Jenkinsfile
Implementation:	Workflow and evidence Build the Docker image from Jenkins The pipeline checked out the repository and executed the Docker image build. 
Step 1	

Figure 14.1: Build the Docker image from Jenkins

Step 2

Verify CI/CD stages

The Jenkins stage view confirms successful checkout, image build, and validation.

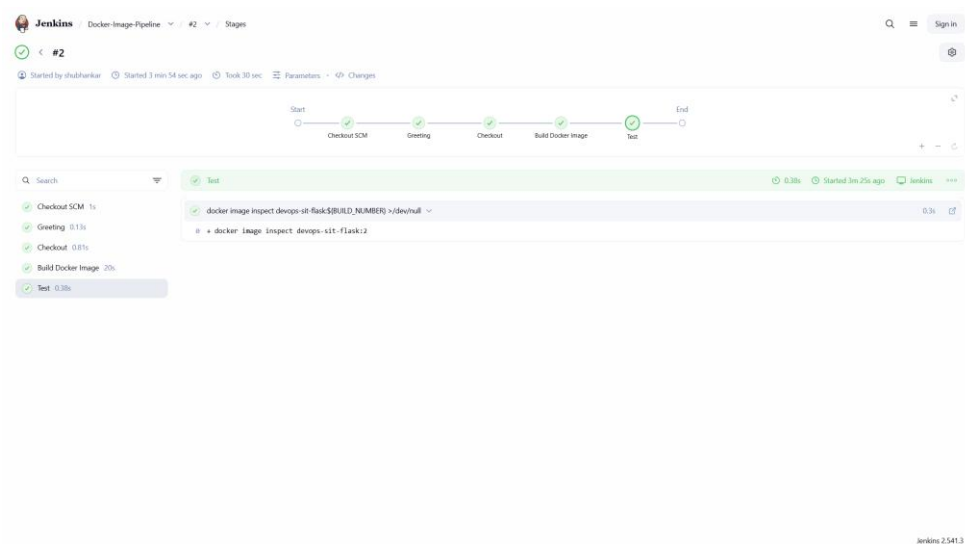


Figure 14.2: Verify CI/CD stages

Result:

The assignment workflow completed successfully, and the screenshots above were captured from the student's implementation.

Subject Faculty Name: Prof. Arijit Dutta



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 15

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Shubhankar Sarangi
PRN No.	23070122206
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025-26, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	Basic Infrastructure as Code with Terraform
Theory:	Key concepts • Infrastructure as Code describes resources in version-controlled configuration instead of manual procedures. • Terraform uses providers to manage resources and maintains state to compare configuration with real infrastructure. • terraform init installs providers, plan previews changes, apply converges the state, and destroy removes managed resources. • An update plan demonstrates idempotent change management, while the final destroy proves lifecycle cleanup. Commands / configuration used • terraform -chdir=terraform init • terraform -chdir=terraform fmt -check • terraform -chdir=terraform validate • terraform -chdir=terraform plan • terraform -chdir=terraform apply -auto-approve • terraform -chdir=terraform destroy -auto-approve
Implementation:	Workflow and evidence
Step 1	Initialize and validate Terraform Terraform installed the required local provider and validated the configuration.

```

● ● ● PowerShell - Assignment 15 - Terraform (1/5)

PS C:\Users\Shubhankar\Downloads\SIT\FinalYCS\devopslab\terraform> terraform version
Terraform v1.16.4
on windows_amd64

Your version of Terraform is out of date! The latest version
is 1.16.5. You can update by downloading from https://developer.hashicorp.com/terraform/install

PS C:\Users\Shubhankar\Downloads\SIT\FinalYCS\devopslab\terraform> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/local versions matching "~> 2.5"...
- Installing hashicorp/local v2.9.1...
- Installed hashicorp/local v2.9.1 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

PS C:\Users\Shubhankar\Downloads\SIT\FinalYCS\devopslab\terraform> terraform validate
Success! The configuration is valid.

PS C:\Users\Shubhankar\Downloads\SIT\FinalYCS\devopslab\terraform> terraform plan -out devops.tfplan

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
+ create

```

Figure 15.1: Initialize and validate Terraform

Review the execution plan

The plan described the local file resource before any change was applied.

[illegible]

Figure 15.2: Review the execution plan

Step 2

Apply and verify the resource

Terraform created the managed file and displayed the configured output.

[illegible]

Figure 15.3: Apply and verify the resource

Update the managed infrastructure

A changed configuration produced an update plan and was applied through Terraform.

```

● ● ● PowerShell – Assignment 15 – Terraform (4/5)

plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # local_file.devops_lab must be replaced
-/+ resource "local_file" "devops_lab" {
  ~ content              = "DevOps Infrastructure created using Terraform by Shubhankar Sarangi." -> "DevOps
Infrastructure updated and managed using Terraform by Shubhankar Sarangi." # forces replacement
  ~ content_base64sha256 = "nwIa1x7dKcfV096cqJx873920qt0rgwJ/Dpn0euCGIs==" -> (known after apply)
  ~ content_base64sha512 = "LwfSxNV3JQGkFtVmcXStyD40i1vGHmUFPxVpQd9Qit7P13aQXTcC6m0b3IGP7XnCSXpW54niTMSA7Rp2Cnygf==" ->
(known after apply)
  ~ content_md5          = "d1002a29121bcee24a9a8da2990a5c35" -> (known after apply)
  ~ content_sha1         = "12f9bdf7ec2b110408a1d70864cfid2b05d5b36" -> (known after apply)
  ~ content_sha256       = "9f02008b1edd29c7d5d3de9c423c41ef7f59d2ab74ae0c09fc3a67d1eb821ac2" -> (known after apply)
  ~ content_sha512       = "9707ec5cd57725018a7c85660974adc83e34d6f18799414f5ef3d877d408b7b3f5dda4174c28026fa9b46f72063fb
5e70b9595b9e2789331203b469d829f27e" -> (known after apply)
  ~ id                  = "12f9bdf7ec2b110408a1d70864cfid2b05d5b36" -> (known after apply)
  # (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.
local_file.devops_lab: Destroying... [id=12f9bdf7ec2b110408a1d70864cfid2b05d5b36]
local_file.devops_lab: Destruction complete after 0s
local_file.devops_lab: Creating...
local_file.devops_lab: Creation complete after 0s [id=3d5b29f0bf8c3026e003153813978f75cbf6c70]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

created_file = "./devops_output.txt"

PS C:\Users\Shubhankar\Downloads\SIT\Final\YCS\devopslab\terraform> Get-Content devops_output.txt
DevOps Infrastructure updated and managed using Terraform by Shubhankar Sarangi.

PS C:\Users\Shubhankar\Downloads\SIT\Final\YCS\devopslab\terraform> terraform destroy -auto-approve
local_file.devops_lab: Refreshing state... [id=3d5b29f0bf8c3026e003153813978f75cbf6c70]

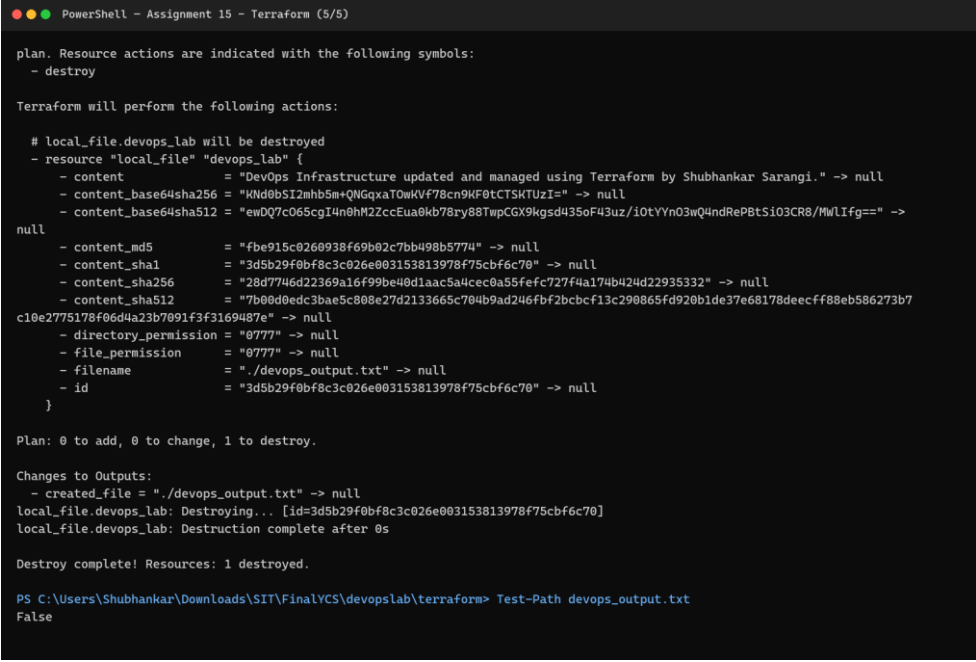
Terraform used the selected providers to generate the following execution

```

Figure 15.4: Update the managed infrastructure

Destroy and verify cleanup

Terraform removed the managed resource and the final filesystem check confirmed its deletion.

	 <pre> PowerShell - Assignment 15 - Terraform (5/5) plan. Resource actions are indicated with the following symbols: - destroy Terraform will perform the following actions: # local_file.devops_lab will be destroyed - resource "local_file" "devops_lab" { - content = "DevOps Infrastructure updated and managed using Terraform by Shubhankar Sarangi." -> null - content_base64sha256 = "Kld0bSI2mhb5m+QNGQxaT0wKVf78cn9KF0tCTSKT0zI=" -> null - content_base64sha512 = "ewDQ7cO65cgI4n0hM2ZccEua0kb78ry88TwpCGX9kgsd435oF43uz/i0tYYn03wQ4ndRePBtSi03CR8/MwLIfg==" -> null null - content_md5 = "fbe915c0260938f69b02c7bb498b5774" -> null - content_sha1 = "3d5b29f0bf8c3c026e003153813978f75cbf6c70" -> null - content_sha256 = "28d7746d22369a16f99be40d1aac5a4cec8a55fetc727f4a174b424d22935332" -> null - content_sha512 = "7b00d0edc3bae5c808e27d2133665c704b9ad246fbf2bcbcf13c290865fd920b1de37e68178deecff88eb586273b7c10e2775178f06d4a23b7091f3f3169487e" -> null - directory_permission = "0777" -> null - file_permission = "0777" -> null - filename = "./devops_output.txt" -> null - id = "3d5b29f0bf8c3c026e003153813978f75cbf6c70" -> null } Plan: 0 to add, 0 to change, 1 to destroy. Changes to Outputs: - created_file = "./devops_output.txt" -> null local_file.devops_lab: Destroying... [id=3d5b29f0bf8c3c026e003153813978f75cbf6c70] local_file.devops_lab: Destruction complete after 0s Destroy complete! Resources: 1 destroyed. PS C:\Users\Shubhankar\Downloads\SIIT\FinalVCS\devopslab\terraform> Test-Path devops_output.txt False </pre> <i>Figure 15.5: Destroy and verify cleanup</i>
Result:	The assignment workflow completed successfully, and the screenshots above were captured from the student's implementation.

Subject Faculty Name: Prof. Arijit Dutta